

A TUTORIAL ON GENAI AND SIMULATION MODELING INTEGRATION

Mohammad Dehghani, Sahil Belsare, and Negar Sadeghi

Department of Mechanical and Industrial Engineering

Northeastern University

360 Huntington Ave, Boston, MA 02115, USA

ABSTRACT

This tutorial provides insights into how GenAI and Large Language Models can support every stage of the simulation modelling workflow, from problem framing to evaluation. The aim is to clarify what these models do well in simulation today, what they still get wrong, and how to use them without giving up rigour. Building on the 2025 edition, this revision adds the Model Context Protocol as an agentic construction route, a heuristic versus LLM-in-the-loop decision-rule comparison inside a running model, and a fully reproducible repository.

1 INTRODUCTION

Some of the most revolutionary products appear to be conceived in a single night, yet they are almost always built on decades of quiet, accumulating research. The simulation modelling community knows this story well. What looks today like a mature, multi-method discipline is the product of more than half a century of incremental progress, from the first simulation languages of the 1960s (GPSS, SIMSCRIPT, SLAM), through the GUI revolution of the 1990s (ProModel, Arena), to the multi-method platforms of the 2000s such as AnyLogic, FlexSim, and Simio (Figure 1).

Artificial intelligence has followed a strikingly parallel arc: rule-based and expert systems in the 1960s, classical machine learning in the 1990s, deep learning in the 2010s, and GenAI in the early 2020s.

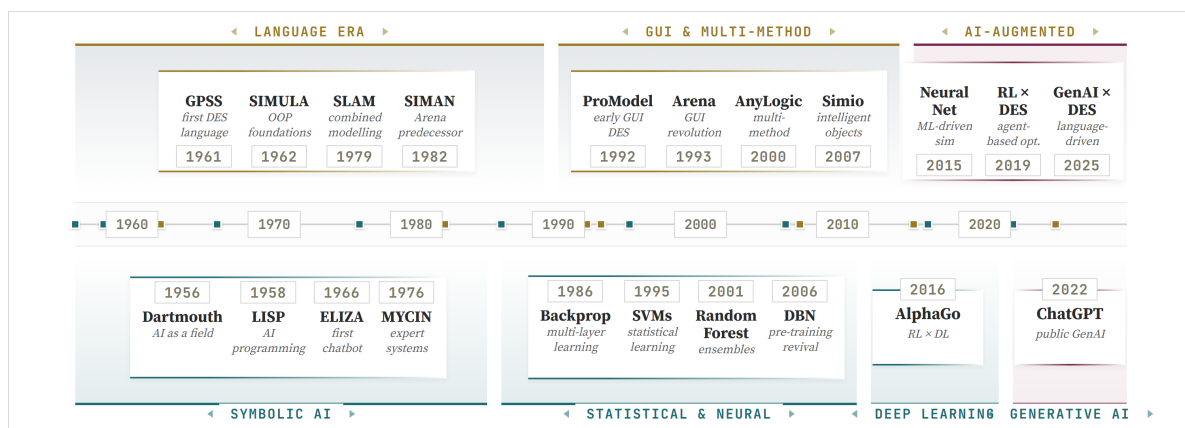


Figure 1: Notable milestones in both simulation modelling and artificial intelligence.

GenAI and Large Language Models (LLMs) have moved rapidly from research artefacts to general-purpose tools used across software, scientific, creative, and decision-making work (M. Zao-Sanders 2025; Dehghanimohammadabadi and Kabaday 2026). The integration of LLMs with simulation modelling, however, did not gain the same momentum until recently. The shift came as frontier models grew capable of producing runnable code on technical problems (Chen et al. 2021; Rozière et al. 2024), at which point the same model that drafts a simulation model can also narrate its output and reason about its behaviour.

This paper continues the 2025 edition of the tutorial (Dehghani et al. 2025), which introduced a capability framework for LLM use across the discrete-event simulation workflow. The pace of change since that edition has been notable. Frontier models now write executable simulation code on the first attempt where iteration used to be required, the Model Context Protocol (Anthropic 2024) has given agentic construction a standard interface that was impractical only a year ago, and the community has shifted from conceptual tutorials toward reproducible artefacts that readers can run, fork, and extend. The present revision tracks these shifts and is the source of the new material throughout the paper.

This paper makes three contributions:

- A revised tutorial that restructures the 2025 framework around five workflow stages (Problem Formulation, Creation, Execution, Experimentation, and Evaluation), each containing three contributions of increasing LLM complexity.
- A working agentic AI workflow built on the Model Context Protocol that allows language models to construct simulation models by calling exposed simulation primitives, rather than generating monolithic source files.
- An empirical cross-model comparison of two frontier large language models acting as decision agents inside a running simulation (bed assignment with the LLM-in-the-loop), against deterministic heuristic baselines.

All artefacts that produced these results, including the SimPy model, prompt templates, evaluation harness, and a live demonstration in the browser, are released as a public companion repository.¹

1.1 Running example: Emergency Department

A multi-stage emergency department (ED) model serves as the single running case study throughout this paper. The purpose of using one consistent example is pedagogical, allowing us to showcase how the LLM contributes at each stage of the simulation workflow without continually switching contexts. The system models patients arriving at the ED, being triaged into severity bands, possibly passing through imaging or laboratory services, and ultimately being admitted to inpatient beds or discharged. The same case is later used to anchor two head-to-head contrasts in the paper, vibe-coding against structured construction at the Creation stage, and a heuristic policy against an LLM-in-the-loop policy at the Execution stage.

2 LARGE LANGUAGE MODELS

GenAI has reshaped the technology landscape in the four years since ChatGPT’s public release in late 2022, and at the centre of that change sits a single architecture, the large language model. An LLM predicts the next token in a sequence using the Transformer architecture (Vaswani et al. 2017), which replaced the strictly sequential processing of older recurrent networks with self-attention. Three steps run in order: the input string is tokenized and each token is mapped to a dense vector with a positional encoding (Figure 2a); a stack of Transformer blocks repeatedly applies multi-head self-attention and feedforward layers; and the final layer produces a probability distribution over the vocabulary, from which the next token is sampled (Figure 2b). Three parameters control how much variation that sample tolerates. *Temperature* rescales the distribution before sampling: low values concentrate mass on the most likely token, high values flatten the distribution (Figure 3). *Top-k* restricts the sampling pool to the k highest-probability tokens. *Top-p*, also known as nucleus sampling, keeps the smallest set of top tokens whose cumulative probability exceeds a threshold p , which adapts to the local shape of the distribution. Throughout this paper we run with temperature 0.2 for code-generation prompts and 0.7 for narration prompts, with top-p 0.9 in both cases.

¹Source and live demo: <https://github.com/mdehghani86/wsc26-genai-sim> and <https://mdehghani86.github.io/wsc26-genai-sim/>.

Our earlier tutorial (Dehghani et al. 2025) treats this material at full length and is the recommended primer for readers who want the lower-level view.

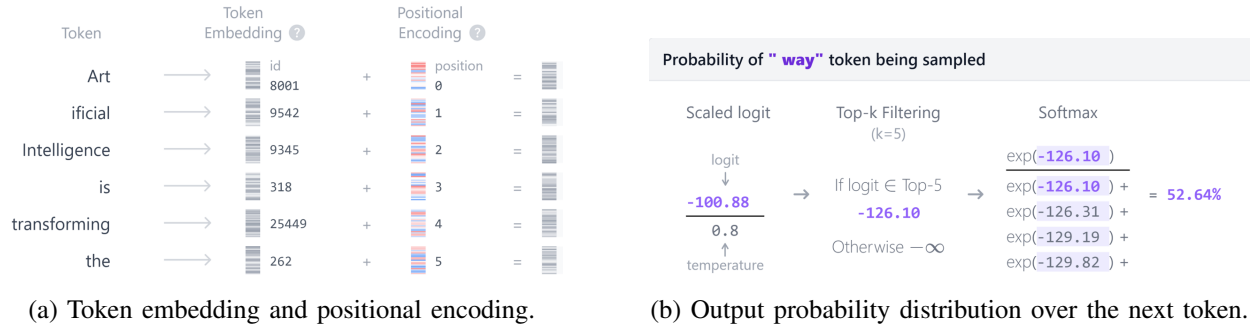


Figure 2: First and last steps of a Transformer pass.

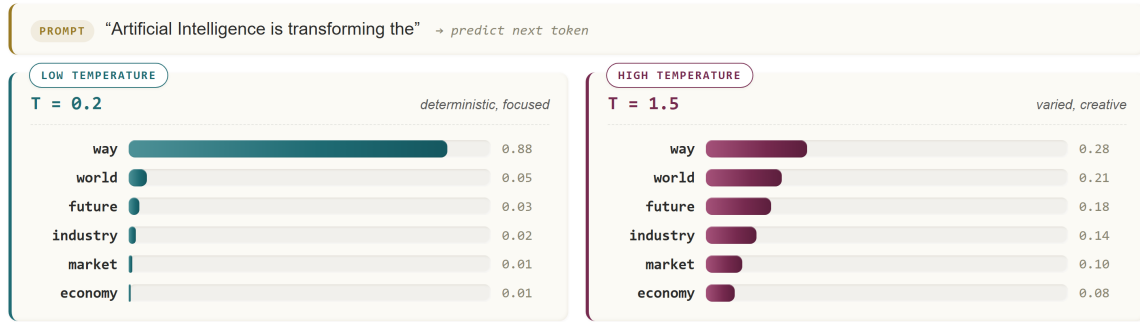


Figure 3: Same prompt under two temperatures: $T=0.2$ concentrates probability on a single token; $T=1.5$ flattens the distribution.

3 GENAI-DRIVEN SIMULATION FRAMEWORK

The paper organises GenAI-assisted simulation into five typed stages, each containing three rising-complexity LLM capabilities (Figure 4). The five stages trace the lifecycle of a simulation study: *Problem Formulation* captures what is being modelled, *Creation* turns the brief into runnable code, *Execution* runs the model and supports in-simulation decisions, *Experimentation* explores scenarios, and *Evaluation* closes the loop with reproducibility and benchmarking. A five-check validation gate at the end of Creation prevents broken models from propagating downstream.

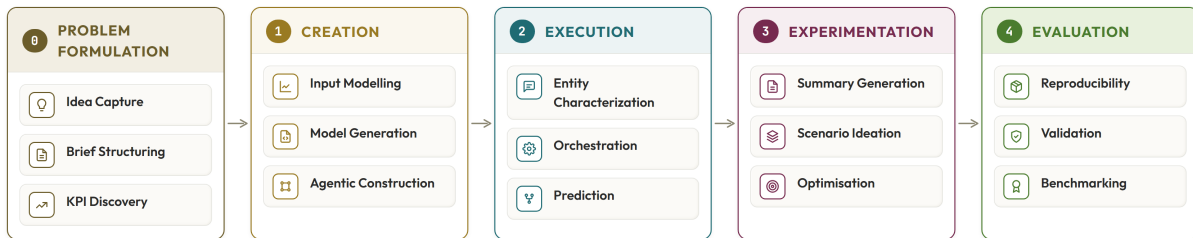


Figure 4: The GenAI-driven simulation framework with five stages, each contributing three rising-complexity LLM capabilities.

4 STAGE 0: PROBLEM FORMULATION

Problem formulation is the first and most consequential step of a simulation study. Without a clear definition of what is being modelled, what the scope is, and which metrics will judge success, the cleanest code and the most rigorous statistics still produce the wrong answer. This is where GenAI is already known to add the most value in professional practice: ideation, structuring, and surveying what others have done. The Problem Formulation stage applies these strengths to simulation through three rising-complexity LLM contributions:

- **Idea Capture.** The LLM is prompted in natural language with the study question and produces a candidate framing. This is the simplest contribution, a rephrasing rather than a verdict.
- **Brief Structuring.** The candidate framing is forced into a fixed three-row schema (problem, scope, KPIs) so that downstream stages consume the artefact verbatim.
- **KPI Discovery.** The LLM is asked to consult recent literature and propose KPIs that reflect community practice, rather than the modeller’s prior habits, for the third row of the schema.

Recent analyses of professional GenAI adoption show that ideation, problem framing, and quick literature scans are among the most widely used everyday capabilities of these models (M. Zao-Sanders 2025). Simulation studies depend on exactly the same kinds of activities at the very start, where the modeller has to articulate the question, fix the boundary of the system, and choose the metrics by which the study will be judged. The Problem Formulation stage operationalises this overlap. The LLM is asked to fill a fixed three-row brief on problem, scope, and KPIs, and that brief is the artefact the rest of the paper carries forward unchanged. All prompts in this paper were issued to Anthropic’s Claude² and reproduced verbatim from the chat transcript.

For the ED running example shown in Figure 5, the prompt states the request in ordinary language, ties it to the specific facility and study question, and asks the LLM to consult recent literature so that the KPIs reflect community practice rather than the modeller’s prior habits. The response is forced into a fixed three-row schema. That structure makes outputs comparable across projects and across language models, and it reveals weak cells early, usually in the scope and boundaries row, before a long script is written against an unstated assumption. The caution from Si et al. on LLM ideation still applies here (C. Si and D. Yang and T. Hashimoto 2024), but the schema turns recombined published practice into an artefact the later stages can use.

PROMPT	
I am planning a discrete-event simulation of an emergency department. Help me formulate the study as a structured problem statement.	
NOTES	small community hospital, two triage tiers, the team wants to evaluate staffing and routing rule changes.
SEARCH	recent literature (2023–2025) on emergency department simulation studies and pull common KPIs.
RETURN	a single table with three rows: <i>Problem definition</i> , <i>Scope & boundaries</i> , and <i>KPIs</i> . Keep each cell to two or three short sentences.

RESPONSE	
ELEMENT	STATEMENT
1 Problem definition	Reduce length of stay in the emergency department and identify whether triage-rule changes or staffing changes deliver the larger improvement under fluctuating arrival rates.
2 Scope & boundaries	From walk-in arrival to admit-or-discharge. Two triage tiers (urgent, non-urgent), a shared imaging and laboratory pool, and fixed bed pools. Ambulance routing, no-shows, and inpatient bed flow are out of scope.
3 KPIs	Mean and 95th-percentile length of stay; door-to-doctor time per triage tier; bed and imaging utilisation; daily throughput.

Figure 5: Problem-formulation prompt and response: a fixed three-row schema (problem, scope, KPIs) that downstream stages consume unchanged.

The three rows of the brief are not a deliverable in isolation. Each row drives directly into one of the three downstream stages of the workflow. The scope row determines which resources and processes

²Specifically, claude-opus-4-7. The empirical in-simulation comparison of Section 8 additionally exercises GPT-5.1 and Gemini-2.5-flash.

appear in the model produced by Creation (Section 5), so any element it omits, ambulance routing for example, cannot reappear later in the simulation or in the narration. The problem-definition row sets the question that Execution (Section 6) must ultimately answer, since a trace that does not address it has run but produced no evidence. The KPI row supplies the success criteria of Experimentation (Section 7) and the targets against which the cross-model benchmark of Evaluation (Section 8) is judged.

5 STAGE 1: CREATION

Model creation is typically the most time-consuming step of a simulation study, since the modeller carries the full burden of specifying the model’s logic, sequencing, and structural decisions. Commercial discrete-event simulation packages are closed by design, which limits how directly external tools can be brought to bear on that step. Open Python ecosystems such as SimPy have grown over the past decade into a usable target for general code-generation tooling, so a frontier LLM that writes Python code can plausibly help with model construction. This motivates the vibe-coding workflow used throughout this stage. The following subsections discuss how GenAI can support simulation model creation from three perspectives.

5.1 Input Modelling

Input modelling has been a careful statistical exercise long before language models entered the simulation workflow. The activity takes observed system data and produces the probability distributions and random-variate generators that the simulation will sample from. Well-established statistical tests, including Kolmogorov–Smirnov, Anderson–Darling, and binned chi-square, together with Q-Q plots for visual diagnostics, are used to judge whether a candidate distribution fits the data.

A language model can step into this pipeline in three distinct ways, each at a different level of responsibility.

- **(i) Fully delegated to the LLM.** Hand the LLM the raw data and let it decide. This is the lightest pattern to set up. Recent benchmarks tighten what to expect from it. [Song et al. \(2025\)](#) evaluate ChatGPT and Llama variants on a curated set of statistical analysis tasks and find that the models produce syntactically valid code but struggle on tasks requiring deeper domain understanding, while [Li et al. \(2025\)](#) report that more than twenty state-of-the-art LLMs still have significant room for improvement on complex real-world table reasoning. In practice the verdict the model returns is also opaque unless the prompt forces an explicit method, and accuracy on large or heavy-tailed traces is uneven.
- **(ii) LLM as candidate proposer.** Use the LLM as a candidate-distribution proposer, with the heavy statistical work happening before and after the LLM call. The pipeline summarises the data into a small set of moments and a histogram description, asks the LLM for a handful of candidate continuous distributions with one rationale each, fits those candidates programmatically, and runs the goodness-of-fit tests against the empirical data. The general principle is to use the LLM only at the ends of the pipeline, proposing candidates and writing the verdict, and let standard goodness-of-fit tests have the final word in between.
- **(iii) LLM-generated pipeline.** Use the LLM to write the input-modelling tool itself. The Python script that loads the trace, fits candidates, runs the tests, and produces the Q-Q diagnostic is a routine piece of scientific Python that frontier LLMs can draft and refine with little supervision.

To demonstrate the framework on the running ED case, this paper combines approaches (ii) and (iii). The analysis tool itself was vibe-coded, and the runtime pattern it implements is candidate proposal. The companion repository ships the result as a live in-browser pipeline that takes the data, computes summary statistics and a histogram, asks the LLM for two or three candidate distributions, fits and tests them against the empirical data, and lets the LLM narrate the verdict.

Figure 6 illustrates the failure mode this pipeline is designed to catch. On a deliberately bursty arrival trace, all three of the LLM’s candidates are rejected at the 5% level. The empirical histogram and the LLM proposals look reasonable in isolation, but the Q-Q plot and the test column make the failure visible. The trace is non-stationary, mixing rush hours with off-peak periods, and no single marginal distribution can absorb that variation. The verdict at the bottom of the figure is what the LLM writes after reading the test output, and its recommendation, refit per hour-of-day, matches what classical input-modelling practice would have suggested. The point is not that the LLM was wrong, but that the test is what made the failure visible.

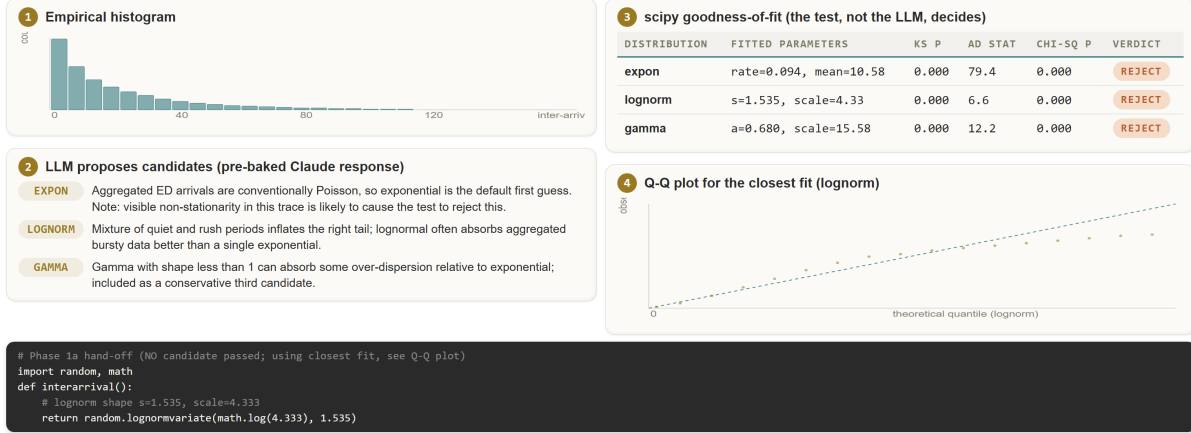


Figure 6: Candidate-proposal input modelling pipeline applied to an ED arrival trace.

Input modelling produces two artefacts that travel together to Model Generation (Section 5.2). One is the fitted random-variate generator, the other is the verdict from the goodness-of-fit tests. Carrying the verdict forward means the next stage knows when a fit was rejected, and cannot silently assume a distribution the data did not support.

5.2 Model Generation

Simulation model creation requires the modeller to encode a long list of structural claims about a real system, including the capacity of each resource, the queueing discipline at each station, the routing logic between stations, the distributions that govern arrivals and service times, and the consistency of all of these choices with one another across the whole file. Each claim has to remain valid as the model evolves, because a single inconsistency can quietly invalidate every KPI the model produces. This density of interlocking decisions is what makes model creation the most demanding of GenAI’s possible roles inside a simulation study.

Modern LLMs are capable of most of what model generation requires. They can write working code in libraries they have not previously seen, follow long structured prompts without losing the thread, and pick up implicit conventions from a handful of examples, capabilities that map cleanly onto simulation modelling as a code-generation task. Open simulation ecosystems such as SimPy help by exposing every primitive directly in Python, so the same kind of code an LLM uses to produce a working web endpoint can in principle express an M/G/c queue, an emergency-department flow, or a small manufacturing line. The case is supported by results from the recent literature. [Chen et al. \(2026\)](#) present a category-template framework that guides LLMs toward mechanism-faithful SimPy queueing models using staged adaptation, which is reason enough to expect that frontier-model prompting is a defensible default for a tutorial reader.

A simulation file that compiles and runs is not the same as one that is correct. Two generations from the same prompt can both execute cleanly and both return plausible KPIs, yet one of them quietly violates

an invariant the modeller never wrote down. Two mechanisms close this gap in the case study below on the running emergency department example, a structured prompt and an automated validation gate.

- **Structured prompt.** The prompt fixes the LLM’s task framing and is re-issued mechanically whenever the upstream brief changes. The template, introduced in our earlier tutorial, has five rows for role, goal, scenario, inputs, and output. The scenario row is copied from the Stage 0 brief verbatim, the inputs section reuses the random-variate generator from §5.1, and the rest is mechanical.
- **Automated validation gate.** The gate is the mechanism that does the substantive work, since silent bugs are hard to catch on a casual read. The companion implementation places a fixed five-check gate between the generated file and the downstream stages (Figure 7). The first three checks are structural, confirming that the entity flow matches the brief end to end, that every producer wires into its declared consumer with no orphan resources, and that a resource pool with capacity c actually serves c entities in parallel rather than collapsing to one. The parallelism check is worth a sentence on its own, since the canonical silent failure here is a single per-pool “manager” process that funnels every job through one server even when `Resource(capacity=c)` is declared correctly. The remaining two checks are statistical and theoretical, verifying that sampled service times match the input-modelling fit in mean, variance, and support, and comparing realised utilisation, mean queue length, and mean waiting time against closed-form Erlang-C and Allen–Cunneen predictions for the matching $M/G/c$ system.

A failed check is fed back into the next prompt so the LLM can fix the affected section directly, and three failed rounds escalate the case to the agentic-construction route of Section 5.3. Creation hands over a simulation file and its passing validation report as a single artefact, since nothing reaches the downstream stages without one.

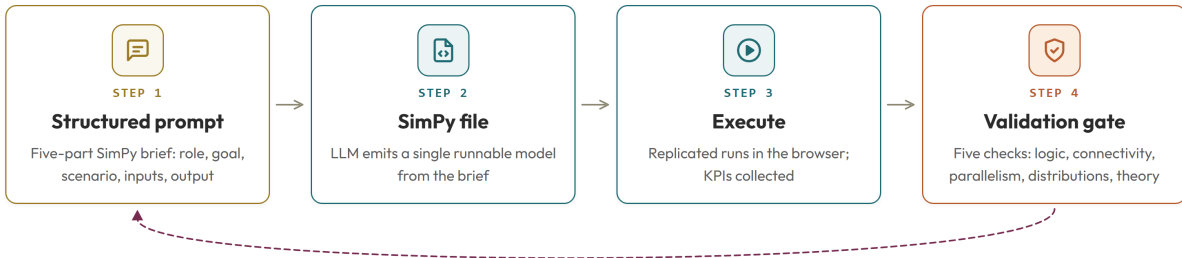


Figure 7: Vibe-coding loop: a structured prompt yields a SimPy file that is executed and screened by a five-check validation gate; any failing check is fed back into the next prompt.

The validated simulation file, with its passing report, is published in the companion repository (Section 8); Execution and Experimentation consume it without further modification.

5.3 Agentic Construction

Vibe coding behind a validation gate is one way to let an LLM author a simulation, and it works at tutorial scale. The interaction pattern has a built-in limit, however. Because the model emits a complete file in one shot and the gate passes or rejects it as a whole, any failure, however local, restarts the entire emission, and the diagnostic the model receives is the failed check rather than the line that produced it. The general alternative is to constrain the LLM to construct the model through a small set of typed actions and let the surrounding tool layer enforce invariants on every call.

The Model Context Protocol is an open standard, introduced by Anthropic in November 2024, for connecting language models to external tools and data sources (Anthropic 2024). An MCP *server* exposes

three primitives: tools (functions the LLM can call), resources (read-only data the LLM can request), and prompts (parameterised templates). An MCP *client*, embedded in the LLM host application, discovers the server’s capabilities at session start and routes the model’s tool calls to the server over JSON-RPC. The protocol has been the subject of two recent surveys, one focused on MCP itself (Singh et al. 2025) and one positioning it within the broader landscape of agent interoperability protocols (Ehtesham et al. 2025).

The relevance to simulation is direct. A discrete-event simulation model is in essence a small set of resource declarations and a small set of process definitions. If those operations are exposed as MCP tools, the LLM no longer needs to write a complete code file in one shot. Instead, it issues typed calls to define resources, define processes, wire producers to consumers, run replications, and query KPIs, and the server enforces the relevant invariants on every call. Invalid arguments and unsatisfied dependencies are rejected at call time rather than after the entire file is regenerated, and theory checks running alongside the simulation produce diagnostics the LLM can read and act on without rewriting the whole model. The construction of the model becomes a structured dialogue with a typed API rather than a bet on a one-shot code emission.

The main advantage of this route in DES is that feedback arrives at the level of each construction step. Instead of emitting a complete file, the model defines resources and processes, connects producers to consumers, requests a run, and queries KPIs through typed calls. If a connection is wrong, the server returns a typed error and the model can repair that single step. If a theory check disagrees on utilisation, the diagnostic points to the parameter at fault rather than to the file in general. The companion repository therefore includes a minimal simulation MCP server with resource definition, process definition, wiring, replication runs, and KPI queries, and the ED example is assembled through that interface across triage, imaging, laboratory, beds, arrivals, and admit or discharge flow.

The two routes are complementary rather than competing. Vibe coding with a validation gate suits a modeller who wants a single file to read, version, and run outside any agent loop; agentic construction suits incremental authoring, situations where invariants must be enforced at construction time rather than after the fact, and briefs that will be re-run against shifting parameter sets. Recent work confirms the trade-off from different angles: Vasa and Sarjoughian (2025) take the structured construction route via PDEVs-LLM with a statechart-driven workflow, and Kleiman et al. (2025) present a general simulation-agent framework that decouples natural-language scenario configuration from model execution. Our contribution is the side-by-side comparison itself, with both routes shipped in the companion repository and evaluated on dimensions agreed in advance: iteration count, first-pass executability, and diagnosability of failures.

6 STAGE 2: EXECUTION

During execution, a language model can move from author of the simulation to participant in it. What it can do during a run is different in kind from what it does at design time, and the simulation modeller now has a small set of new collaborators to think about rather than a single code-writing assistant. The first of these collaborators, *Entity Characterization*, comes into play at the boundary where new entities arrive, when their attributes are still soft and a language model can give them richer descriptions than a numeric draw allows. A second, *Orchestration*, lives at the recurring decision points along an entity’s path, replacing the deterministic rule that has historically lived in code with a policy that reads the live state and reasons about it. The third, *Prediction*, looks the other way and reads back over the trace the run has already produced, forming an expectation about what is coming next that can in turn feed the previous two. The Execution stage takes these three in that order, from the lightest contribution to the most demanding.

Across the three contributions, a common pattern emerges. The language model is not asked to take over the simulation, only to step into specific moments where the modeller would otherwise have to encode something by hand or accept a fixed default. Each contribution is therefore judged by what it adds to the simulation rather than by how much of the simulation it replaces, and the case study that anchors each subsection is designed with that division of labour in mind.

6.1 Entity Characterization

Most simulation models initialise their entities with a fixed set of numeric attributes, an arrival time, a severity band, a service-time draw, chosen by the modeller at design time. Real systems carry far more information at the same moment of entity creation, and much of it is unstructured. LLMs are unusually well placed to convert that information into structured attributes that the rest of the model can act on. A patient checking into an emergency department arrives with a free-text complaint that an LLM can summarise into a triage severity, an estimated complexity, and a flag for whether imaging is likely to be needed. A customer arriving at a service centre can be tagged with intent and urgency from the wording of their request. A truck arriving at a port can be classified by cargo type from the booking note. The general capability is the same one that frontier models exhibit on classification and information-extraction tasks elsewhere, brought into the simulation at the point where attributes are first assigned.

The simplifying assumption that makes this contribution easy to deliver is that characterization happens once, at the moment of entity creation, rather than continuously during the run. The simulation does not need to call an LLM mid-run, and the only API calls happen offline as part of a pre-processing step that produces a static lookup keyed by entity. The companion repository ships such a lookup for the running emergency-department example, generated from a small set of synthetic patient backstories, and the simulation reads attributes from it as new patient entities are spawned. The methodological point of this subsection is that an LLM can move information that the modeller would otherwise have to encode by hand into a form the model can consume, without the latency, cost, or reproducibility issues that come with calling the LLM live.

6.2 Orchestration

The next capability up the ladder is orchestration: letting the LLM act as the decision-maker at recurring choice points during the simulation. A discrete-event model typically contains a small number of such decisions (which server to assign to, which order quantity to release, which route to take), and each one is conventionally expressed as a deterministic rule of the current state. Letting a language model take that role replaces the rule with a context-aware policy that can change its reasoning across replications, but introduces three failure modes worth naming up front: per-decision latency on the order of hundreds of milliseconds, non-reproducibility unless temperature and prompts are pinned, and the absence of any automatic variance-reduction reasoning.

Recent work has demonstrated the pattern in adjacent simulation domains. [Y. Quan and Z. Liu \(2024\)](#) place an LLM agent inside a multi-echelon inventory simulation as the order-quantity decider, and [Wang et al. \(2025\)](#) place one inside a transportation simulation as the route-choice decider. In the commercial DES ecosystem, [Dehghani \(2026\)](#) integrate frontier LLMs with the Simio runtime so the model can both author entity logic and act on simulation state through natural language interfaces. In our tutorial case the recurring decision is bed assignment: each arriving patient must be placed in one of b inpatient beds or held in queue. Three deterministic rules are natural baselines, round-robin, severity-priority with time-in-queue tie-break, and a clinician-style rule that balances bed turnover against remaining length of stay. The LLM-in-the-loop variant is added as a fourth option: at each assignment step the simulation pauses, serialises the relevant state, sends it to an LLM with a short instruction, and uses the returned index. The pseudocode is short enough to fit on a slide:

```
def llm_assign(state, llm):
    """Decide which bed index to use, given the current ED state."""
    prompt = render_prompt(state) # serialise occupancy, severity, queue
    reply = llm(prompt, temperature=0.0, max_tokens=8)
    return parse_bed_index(reply) # int in [0, b)

# Drop-in: the SimPy process calls llm_assign(state, llm) at each
```

admission step instead of the round-robin or severity-priority rule.

For this pattern to behave like a policy, each LLM call needs a compact record of recent decisions rather than an isolated snapshot of the current state. The companion implementation provides that context with a rolling buffer of recent assignments and a short summary of the policy so far, both attached to the next prompt. This design adds wall time because each frontier-model call can cost hundreds of milliseconds or more and that cost accumulates over large numbers of assignments, although caching repeated state vectors and batching calls during trace replay keep the tutorial experiments manageable. It also requires strict replay controls, so temperature is fixed at zero and every prompt and reply is recorded. Under those conditions, the same trace can be run under each heuristic and under the LLM, yielding a KPI table and an agreement-rate diagnostic for the ED comparison that follows.

The implementation keeps this pattern portable across vendors. The same loop can be wired to any frontier model with no change to the simulation, the heuristics ship as built-in baselines for the comparison, and the user supplies an API key only when the LLM variant is being run. Beyond the engineering, the broader point is that a simulation engine able to call out to a language model at decision time is a fundamentally different artefact from one whose decision rule is fixed at compile time, and the trade-offs that come with it (latency, cost, reproducibility, agreement-rate diagnostics) follow directly from that difference.

The bed-assignment case study below exercises this pattern across four admission policies on the same eight paired seeds: *FIFO* (oldest queued patient first), *severity-priority* (lowest severity number first, ties by arrival time), *GPT-5.1 with rolling memory*, and *Gemini-2.5-flash with rolling memory*. Each replication generates 25 patients across 4 beds with severity-stratified log-normal length-of-stay, producing 7–20 admission decisions per replication and 200 LLM calls per model in total. Both LLM policies see the live state, the queue, the last few decisions, and a periodically compacted “policy so far” summary; temperature is fixed to zero, so the only randomness across replications is the simulation seed.

Table 1: Admission-policy benchmark on the ED running example: $N=8$ paired replications, 25 patients each, $B=4$ beds. All wait times in minutes, mean $\pm$ std across replications.

Policy	Mean wait	p95 wait	Sev-1 wait	Sev-3 wait	Util.	Decision lat.
FIFO	28.2 ± 21.2	71.5 ± 33.4	27.9 ± 23.5	30.0 ± 30.6	0.83 ± 0.08	~ 0 ms
Severity-priority	33.0 ± 27.3	112.3 ± 79.0	12.9 ± 8.9	64.1 ± 76.5	0.85 ± 0.09	~ 0 ms
GPT-5.1 + memory	33.0 ± 27.3	112.3 ± 79.0	12.9 ± 8.9	64.1 ± 76.5	0.85 ± 0.09	773 ms
Gemini-2.5-flash + mem.	31.7 ± 24.7	93.5 ± 54.2	12.9 ± 9.0	48.1 ± 50.6	0.85 ± 0.09	585 ms

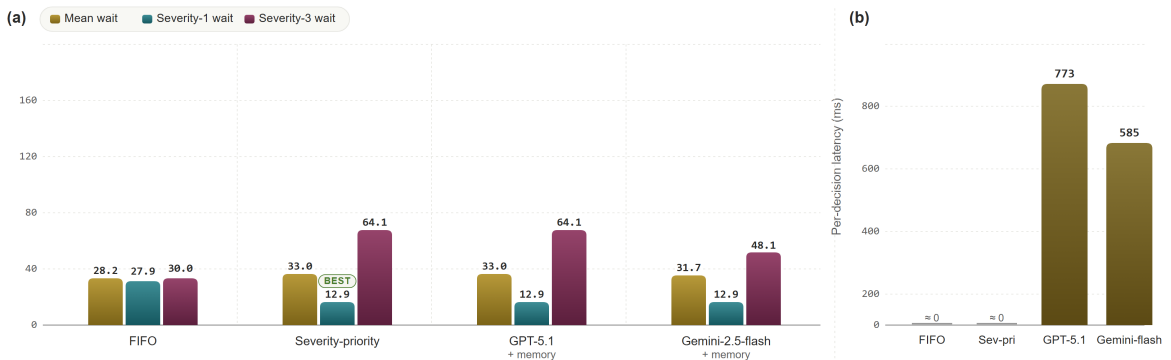


Figure 8: (a) Per-policy wait times; (b) wall-time cost per admission decision.

The results in Table 1 and Figure 8 surface three insights that generalise beyond the ED case. Severity-priority halves urgent-patient wait at the cost of doubling routine-patient wait, a trade-off that an aggregate mean would conceal and that applies to any priority rule which ignores its unpromoted class. GPT-5.1 with memory reproduces severity-priority exactly, which is what the agreement-rate diagnostic from the orchestration pattern is designed to detect, and an LLM that agrees perfectly with a sensible rule is doing what the rule would do. Gemini-2.5-flash with memory, by contrast, deviates on a meaningful fraction of decisions, and the deviations are not random; they concentrate on the longest replications where routine patients begin to starve, producing the contextual hybridisation that a static rule cannot express and that justifies an LLM-in-the-loop variant when state-dependent trade-offs matter. The price of that flexibility is wall-time, with the LLM policies running several orders of magnitude slower per decision than the heuristics, though caching and batched-replay layers in the companion repository bring effective latency back into the heuristic range.

6.3 Prediction

The third execution capability is predictive. Instead of choosing what to do now, the LLM reads the recent simulation history and forecasts what is about to happen, so that downstream decisions can be informed by an expectation rather than by current queue state alone. The input to the prediction step is the recent (time, entity, event) log together with the brief, and the output is a structured forecast that the rest of the simulation can consume. Typical examples include an expected number of severity-1 arrivals in the next hour, a probability that the current ED census will breach a capacity threshold, and an estimated completion time for an order already in process. The general capability is the same one that frontier models exhibit on event-log and time-series tasks elsewhere, reasoning about a short past window and emitting a probable next-step description, brought into a simulation context where the past window is generated by the run itself.

Predictions of this kind are most useful when they feed back into orchestration. The bed-assignment decision becomes materially better when it can weigh the cost of placing a patient against a one-hour forecast of incoming arrivals, since admitting a routine patient now may shut down a bed that an urgent patient will need shortly. The same logic applies to staffing decisions, to inventory reorder triggers, and to any choice whose right answer changes when the next event is anticipated rather than reacted to. Prediction also generalises the older idea of post-run log analysis, where the LLM reads a finished trace and writes a summary of what happened, by moving that same kind of reading inside the run and turning the summary into an input for the next decision. The companion repository carries a small log-summarisation utility that demonstrates the pattern on the running case study: a rolling window of recent admission events is summarised, passed to an LLM with a short forecast prompt, and the resulting prediction is logged alongside the deterministic queue state. Wiring that prediction directly into the bed-assignment decision is a natural next step, and one that the headline experiments do not yet take, since doing so collapses prediction and decision into a single latency budget. The methodological point of this subsection is that the capability is straightforward to demonstrate once the log infrastructure exists, and that it is the natural next layer of complexity above an orchestration policy that reads only the live state.

7 STAGE 3: EXPERIMENTATION

Once the model runs, the modeller has to design scenarios, sweep parameters, and turn the resulting numbers into something a reader can act on. Designing the scenario set, choosing which figures to show, and searching for good operating points are all places where an LLM can shorten the cycle without taking over the methodological choices. The Experimentation stage exposes three rising-complexity LLM contributions: *Summary Generation*, figure selection plus a short written interpretation of results; *Scenario Ideation*, proposing factors and levels from the brief; and *Optimisation*, using the LLM to drive parameter search, with explicit caveats.

7.1 Summary Generation

The most reliable Experimentation contribution from an LLM combines figure selection with narrative interpretation. Drawing plots from a log file is well-handled by deterministic code, so what the LLM is asked to do here is choose which two or three plots best support the comparison the brief calls for, given the KPI list and the scenario table. Because the plots themselves are produced by code rather than by the LLM, the figures stay reproducible from the log file alone; the companion repository ships the plotting utility.

The narrative half plays directly to a general LLM strength: writing a short interpretive summary of a numeric table for a human reader. The LLM is given the KPI rows from the brief, the scenario table, and short captions for the plots, and is asked to write a paragraph addressed to a stakeholder, naming which scenario performs best on which KPI and where the trade-offs lie. The output is reviewed by the modeller before it leaves the loop. The reason for keeping a human in this seat is the same as in input modelling: the model is good at recombining what it sees, but the responsibility for the claim rests with the modeller. Recent work on LLM-assisted result interpretation in adjacent domains supports this division of labour, with the model handling first-draft narrative and the analyst keeping the final word ([Selak et al. 2026](#); [Kleiman et al. 2025](#)).

7.2 Scenario Ideation

Scenario design at this stage does not need to be invented from scratch, because the factors and levels are already implicit in the brief. The KPI row identifies what the comparison is supposed to measure, with examples in the ED study being mean length of stay, 95th-percentile waiting time, and bed utilisation. Movable factors come from the scope row, which here covers bed count, triage staffing, and the hour-of-day arrival pattern. A small full-factorial or fractional-factorial design over those movable factors, replicated to the precision the KPI row demands, produces the data table that the next subsection will read. The LLM's role at this stage is bounded. It proposes a starting design from the brief, the modeller accepts or trims it, and the remainder of the workflow runs mechanically.

7.3 Optimisation

Optimisation is the most ambitious experimentation use of an LLM in this paper, and the evidence here remains limited. A frontier model can propose the next scenario after seeing the results collected so far, which resembles a loose Bayesian sequential design when the response surface is well behaved. Even so, it should not be treated as a replacement for a dedicated simulation optimiser such as OptQuest or for a black-box optimiser such as SMAC. In our setting, the model tends to over-exploit unless the prompt explicitly protects exploration, and it has no principled way to reason about variance reduction or the effect of larger replication counts on standard error. For that reason we use the LLM-driven optimiser only on small designs with inexpensive evaluations and report the full search trace with the reported optimum.

8 STAGE 4: EVALUATION

A simulation study is only as useful as the trust the reader can place in its numbers. The Evaluation stage closes the workflow by re-running everything against reference instruments and by exposing the same harness to multiple frontier LLMs, so that the cross-model claims rest on a uniform measurement rather than a single chat session. Three rising-complexity LLM contributions sit at this stage:

- **Reproducibility.** Every figure, every prompt, and every result is regenerable from the public repository, and the live in-browser site lets a reader exercise the workflow without installing anything.

- **Validation.** Realised KPIs are cross-checked against closed-form $M/G/c$ bounds and against an off-the-shelf commercial reference model (Arena or Simio configured to the same brief), to provide a second-source check that does not depend on the SimPy implementation.
- **Benchmarking.** The general practice of running multiple frontier LLMs through the same harness on the same paired seeds, generalised as a reusable evaluation principle and applied to the admission-policy comparison in §6.2.

Every artefact in this paper is reproducible from the public companion repository, which hosts the validated SimPy model, the input-modelling demos (clean and bursty), the prompts used at each stage, the validation harness, the tool-server used for agentic construction, and the comparison-table run logs. A companion live site lets a reader exercise the workflow without installing anything; the input-modelling pipeline runs end to end in the browser, and the LLM-in-the-loop variants are exposed under a user-supplied API key.

KPI claims throughout the paper are cross-checked against analytical $M/G/c$ bounds (Erlang-C and Allen–Cunneen for the matching multi-server model), using the same validation harness invoked at each stage of the workflow. As a second-source check we additionally reproduce the brief in an off-the-shelf commercial package (Arena or Simio) configured to the same arrival distribution, service-time fits, and resource pools, and report the realised KPIs side by side with the SimPy run.

The Evaluation step that makes vendor comparisons trustworthy is to run every LLM under examination through the same harness on the same paired seeds, so that any difference in the resulting numbers reflects the model rather than the test bed. The bed-assignment results reported in Section 6.2 are a worked instance of this principle, where four admission policies (two heuristic, two LLM-in-the-loop) share a single decision driver and a single seed list. The general practice for a tutorial reader is to fix every part of the run that is not the model under test, to pin temperature to zero, to record every prompt and reply for replay, and to report mean and spread across enough paired seeds for the comparison to survive a sceptical review. The same harness extends to any execution-stage capability the modeller wants to benchmark across vendors, with the orchestration case study above as the template.

9 DISCUSSION

The aim of this paper is to shed light on the joint force of GenAI and discrete-event simulation, and to offer the simulation community a tutorial framework for thinking about where the two meet. Across the five stages of a typical study, problem formulation, creation, execution, experimentation, and evaluation, we have shown how a language model can take on real responsibility at the soft ends of the pipeline while the substantive decisions inside each stage continue to rest on the instruments the field has spent decades refining.

This edition extends the 2025 framework in three coupled ways. Reproducibility is now end to end, with every simulation model, prompt, and result regenerable from the public companion repository and exercisable in the browser without any local installation. Agentic construction joins vibe-coding as a structurally different authoring path, with the LLM emitting typed tool calls against a small simulation API rather than a one-shot file, and the two paths ship side by side on the same brief so the trade-offs are visible. The validation gate that protects model creation also doubles as a uniform scorecard for comparing frontier LLMs, and the pilot bed-assignment benchmark gives a first set of numbers on the ED case. None of these is a finished result, and the companion repository is structured so readers can extend the comparison to their own briefs and their own models.

The field is moving quickly. Frontier models that struggled to draft a single SimPy file two years ago now author entire pipelines on the first attempt, and the tooling around them, from the Model Context Protocol to in-browser execution to vendor-neutral evaluation harnesses, has matured at a similar pace. The simulation community will not stand still in the middle of that change, and the most productive response is to embrace the parts of the toolchain that are now genuinely useful while keeping the methodological

discipline the field is known for. This paper is one step in that direction, and the companion repository is the invitation to take the next.

REFERENCES

- Anthropic 2024, November. “Introducing the Model Context Protocol”. Anthropic Engineering Blog. Accessed April 2026.
- Chen, J.-Q., K. Zhang, R. Zheng, and Y. Zhong. 2026. “Mechanism-Faithful Queueing Simulation Model Translation with Large Language Model Support”. *arXiv preprint arXiv:2601.06543*.
- Chen, M., J. Tworek, H. Jun, Q. Yuan, H. Ponde de Oliveira Pinto, J. Kaplan, *et al.* 2021. “Evaluating Large Language Models Trained on Code”. *arXiv preprint arXiv:2107.03374*.
- Dehghani, M. 2026. “SimGPT: Unlocking Potentials of Generative AI in Simulation via Simio and LLM Integration”. *Computers and Industrial Engineering* <https://doi.org/10.1016/j.cie.2026.112104>. Available online 9 May 2026, journal pre-proof.
- Dehghani, M., S. Belsare, and N. Sadeghi. 2025. “A Tutorial on Generative AI and Simulation Modeling Integration”. In *Proceedings of the 2025 Winter Simulation Conference*, 1197–1211. Seattle, WA.
- Dehghanimohammadabadi, M., and N. Kabadayi. 2026. “The AI-Driven Decision-Making (AIDM) Framework: Integrating AHP and ChatGPT-4 for Supplier Selection”. *Annals of Operations Research* <https://doi.org/10.1007/s10479-026-07136-7>.
- Ehtesham, A., A. Singh, G. K. Gupta, and S. Kumar. 2025. “A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)”. *arXiv preprint arXiv:2505.02279*.
- Kleiman, J., K. Frank, J. Voyles, and S. Campagna. 2025. “Simulation Agent: A Framework for Integrating Simulation and Large Language Models for Enhanced Decision-Making”. *arXiv preprint arXiv:2505.13761*.
- Li, C., X. Liu, Z. Song, C. Chi, C. Zhao, J. Yang, *et al.* 2025. “TReB: A Comprehensive Benchmark for Evaluating Table Reasoning Capabilities of Large Language Models”. *arXiv preprint arXiv:2506.18421*.
- Y. Quan and Z. Liu 2024. “InvAgent: A Large Language Model based Multi-Agent System for Inventory Management in Supply Chains”. *arXiv preprint arXiv:2407.11384*.
- Rozière, B., J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan *et al.* 2024. “Code Llama: Open Foundation Models for Code”. *arXiv preprint arXiv:2308.12950*.
- Selak, M., D. Krechel, A. Ulges, S. Spieckermann, N. Stoehr, and A. Loehr. 2026. “Agentic Insight Generation in VSM Simulations”. *arXiv preprint arXiv:2604.12421*.
- C. Si and D. Yang and T. Hashimoto 2024. “Can LLMs Generate Novel Research Ideas? A Large-Scale Human Study with 100+ NLP Researchers”. *arXiv preprint arXiv:2409.04109*.
- Singh, A., A. Ehtesham, S. Kumar, and T. Talaei Khoei. 2025. “A Survey of the Model Context Protocol (MCP): Standardizing Context to Enhance Large Language Models”. *Preprints.org* 202504.0245.
- Song, X., K. Xie, L. Lee, R. Chen, J. M. Clark, H. He, *et al.* 2025. “Performance Evaluation of Large Language Models in Statistical Programming”. *arXiv preprint arXiv:2502.13117*.
- Vasa, V. K. S., and H. S. Sarjoughian. 2025. “Generative Statecharts-Driven PDEVS Modeling”. In *Proceedings of the 2025 Winter Simulation Conference*. Code at <https://github.com/comses/PDLM>.
- Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, *et al.* 2017. “Attention Is All You Need”. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 5998–6008.
- Wang, L., P. Duan, Z. He, C. Lyu, X. Chen, N. Zheng, *et al.* 2025. “Agentic Large Language Models for Day-to-Day Route Choices”. *Transportation Research Part C: Emerging Technologies* 180:105307.
- M. Zao-Sanders 2025, April. “How People Are Really Using GenAI in 2025”. Harvard Business Review. Accessed April 2026.

AUTHOR BIOGRAPHIES

MOHAMMAD DEHGHANI is an Associate Teaching Professor in the Department of Mechanical and Industrial Engineering at Northeastern University. His research focuses on developing intelligent simulation frameworks integrated with optimization and AI-based models to support decision-making, with applications in healthcare, manufacturing, and supply chain. His email address is m.dehghani@northeastern.edu.

SAHIL BELSARE is a simulation scientist with expertise in digital twin technologies and simulation modeling. He applies machine learning models to enhance decision-making in complex systems, focusing on optimization and reinforcement learning integrated with simulation platforms. His email is belsare.s@northeastern.edu.

NEGAR SADEGHI is a Ph.D. student in Industrial Engineering at Northeastern University. Her research focuses on developing intelligent simulation models for pharmaceutical supply chains and production-distribution systems. Her email address is sadeghi.ne@northeastern.edu.